

Langages de script : python pour initiés

Cours 2 : Idiomatic python, what s the web ?



Un petit aperçu du calendrier

SEM	MARDI	PROGRAMME
39	22/09/2026	Intro et le file system
40	29/09/2026	Pythonismes + Qu'est ce qu'internet
41	06/10/2026	Web crawling and scrapping
42	13/10/2026	Intro datascience + Monitoring Rendu TP1
 43	20/10/2026	Contrôle 1
44	27/10/2026	<i>Vacances</i>
45	03/11/2026	Data visualisation + la récursivité



SEM	MARDI	PROGRAMME
46	10/11/2026	Moteurs de recherche + décorateurs Rendu TP2
47	17/11/2026	La programmation asynchrone
48	24/11/2026	Les API
49	01/12/2026	Intro to AI Agents Rendu TP3
50	08/12/2026	Contrôle 2
51	15/12/2026	Corrigé + le point à faire
52	22/12/2026	<i>Pas de cours</i> Rendu final TP4

- <https://pe8142-0a>



524-47a5-
ue

Interview

Given a string `s`, find the length of the **longest substring** without repeating characters.

Example 1:

Input: `s = "abcabcbb"`

Output: 3

Explanation: The answer is "abc", with the length of 3.

Example 2:

Input: `s = "bbbbbb"`

Output: 1

Explanation: The answer is "b", with the length of 1.

Example 3:

Input: `s = "pwwkew"`

Output: 3

Explanation: The answer is "wke", with the length of 3.

Notice that the answer must be a substring, "pwke" is a subsequence and not a substring.

De quelles structures de données avons nous besoin ?

Décrire une solution en langage naturel.

A quoi faut-il faire attention ?

Approche 1 : trouver l'erreur

```
1 def longest_substring(s) :
2 |
3 |     longest = 0
4 |     current = 0
5 |     current_substring = []
6 |     longest_substring = []
7 |
8 |     # un seul parcours de la chaîne
9 |
10 |    for i, car in enumerate(s) :
11 |        # cas simples : si la nouvelle lettre qu'on lit est différente on allonge notre string
12 |        if car not in current_substring :
13 |            current_substring.append(car)
14 |            current += 1
15 |
16 |        else :
17 |            current_substring = current_substring[i+1:]
18 |            current_substring.append(car)
19 |            current = len(current_substring)
20 |
21 |        if current > longest :
22 |            longest = current
23 |            longest_substring = current_substring
24 |
25 |    print(longest_substring)
26 |    return longest
27 |
```

V2 : code plus succinct

```
1 def longest_substring_2(s):
2
3     char_to_index = dict()
4     left_index = 0
5     longest = 0
6
7     for right, char in enumerate(s):
8
9         if char in char_to_index and char_to_index[char] >= left_index :
10             left_index = char_to_index[char] + 1 # on décale le début de la nouvelle sous chaîne si on repère un doublon
11
12         char_to_index[char] = right
13         longest = max(longest, right-left_index+1 )
14
15     return longest
```

V3 : make it pop

```
1 def longest_substring_3(s):
2     longest = 0
3     q = []
4     for char in s:
5         if char in q:
6             while q.pop(0) != char:
7                 pass #si on repère un doublon on éliminé tout ce qui était avant la chaîne avec pop
8             q.append(char) # sinon on continue de lire la chaîne caractère par caractère
9             longest = max(longest, len(q))
10
11     return longest
```

Python tricks :

Faire des choses éblouissantes en une ligne de code



Python idioms : commen en LV2 élégance

- <https://medium.com/the-andela-way/idiomatic-python-coding-the-smart-way-cc560fa5f1d6>

Idiomatic Python. Coding the smart way.



Seremba John Paul · [Follow](#)

Published in The Andela Way · 5 min read · Sep 5, 2017

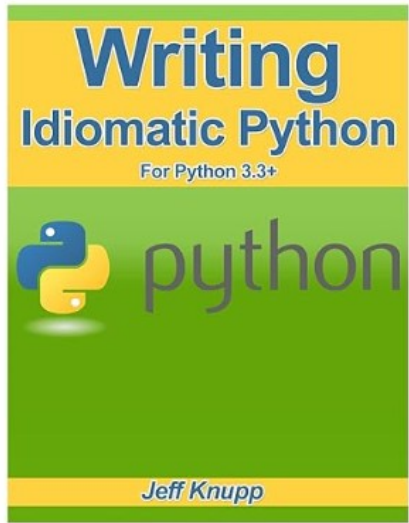


2.3K



19





Writing Idiomatic Python 3.3 Paperback – February 10, 2013

by **Jeff Knupp** (Author)

4.3 ★★★★★ 80 ratings

[See all formats and editions](#)

The "Writing Idiomatic Python" book is finally here! Chock full of code samples, you'll learn the "Pythonic" way to accomplish common tasks. Each idiom comes with a detailed description, example code showing the "wrong" way to do it, and code for the idiomatic, "Pythonic" alternative. *This version of the book is for Python 3. There is also a Python 2.7+ version available.* "Writing Idiomatic Python" contains the most common and important Python idioms in a format that maximizes identification and understanding. Each idiom is presented as a recommendation to write some commonly used piece of code. It is followed by an explanation of why the idiom is important. It also contains two code samples: the "Harmful" way to write it and the "Idiomatic" way. * The "Harmful" way helps you identify the idiom in your own code. * The "Idiomatic" way shows you how to easily translate that code into idiomatic Python. This book is perfect for you: * If you're coming to Python from another programming language * If you're learning Python as a first programming language * If you're looking to increase the readability, maintainability, and correctness of your Python code What is "Idiomatic" Python? Every programming language has its own idioms. Programming language idioms are nothing more than the generally accepted way of writing a certain piece of code. Consistently writing idiomatic code has a number of important benefits: * Others can read and understand your code easily * Others can maintain and enhance your code with minimal effort * Your code will contain fewer bugs * Your code will teach others to write correct code without any effort on your part

[Report an issue with this product or seller](#)

Print length



116 pages

Language



English

Publication date



February 10, 2013

Dimensions



8.5 x 0.28 x 11
inches

ISBN-10



1482374811

Roll over image to zoom in

[Read sample](#)

Follow the author



Jeff Knupp

[Follow](#)

Kindle
EUR 6.79
Available instantly

Paperback
EUR 26.96

[Other Used and New from EUR 21.63](#)

EUR **26⁹⁶**

[FREE International Returns](#)

No Import Fees Deposit & EUR 10.12
Shipping to France [Details](#)
Delivery **Wednesday, October 2**. Order
within **17 hrs 34 mins**

Or fastest delivery **Friday, September 27**

[Deliver to France](#)

In Stock

Quantity: 1

[Add to Cart](#)

[Buy Now](#)

Ships from Amazon.com
Sold by Amazon.com
Returns 30-day refund/replacement
Payment Secure transaction

☐ Add a gift receipt for easy returns

0) La liste par compréhension

Python permet de créer des listes par compréhension, en lui donnant une « recette » pour générer les éléments. C'est joli et très puissant.

Long et pas beau

```
Liste = ["chacal","renard","caribou","chat"]  
Flet = []  
For word in liste :  
    Flet.append(word[0])
```

idiomatique

```
Liste = ["chacal","renard","caribou","chat"]  
Flet = [word[0] for word in liste]
```

1) La gestion du néant

Beaucoup d'objets en python ont par définition une valeur de vérité : une liste vide => Faux, un entier positif => Vrai

Il est donc souvent inutile d'utiliser un booléen dans le code

Peu idiomatique	idiomatique
<pre>if x == True: # do something elif x == False: # do something else</pre>	<pre>If x : # do something Elif not x : # do something else</pre>

-
-
-
-
- NB : avec la valeur None il faut écrire `is None` et non `== None` (singleton)

2) L'attribution efficace

On peut attribuer les valeurs à des variables de façon plus efficace :

Pas idiomatique	idiomatique
a = 2 b = 3	a, b = 2, 3
A = 2 B = 2	A, B = 2 (assignation multiple)
negative = True If negative : var = -1 Else : var = 1	Var = -1 if negative else 1

3) enumerate

- Cette fonction ajoute un itérateur à un objet itérable
syntaxe : `enumerate(sequence, start=0)`

Pas bien	bien
<pre>for i in range(len(l)): x = l[i]</pre>	<pre>for i, x in enumerate(l): # ...</pre>

4) zip

- La fonction zip permet de combiner plusieurs itérateurs en un seul objet. Elle prend en entrée un nombre indéterminé de listes **de même longueur** et en fait transforme cela en une liste de i tuples contenant les ièmes éléments des listes données en entrée

b

```
For i in range(len(liste1)) :  
    El1 = liste1[i]  
    El2 = liste2[i]  
    print(element1, element2)
```

g

```
for element1, element2 in zip(liste1, liste2):  
    print(element1, element2)
```

5) join

Créer une string à partir d'une liste

syntaxe : `element_liant(str).(list)`

bad	good
<pre>S = " " For x in l : S += l</pre>	<pre>s = "".join(f(x) for x in l)</pre>

6) Parcourir un dictionnaire avec items()

- Items est une sorte de zip() sur les couples clé / valeur d'un dictionnaire

b	b
<pre>For k in dic : V = dic[k] Print(k,v)</pre>	<pre>For k, v in dic.items() : print(k,v)</pre>

7) La * dans python

L'astérisque (*) en Python est un opérateur polyvalent qui joue plusieurs rôles selon le contexte.

A) La mutiplication : $a = 3 * 7$

B) multiplication de listes : $liste = [0]*100$

La * dans python

C) Déballage d'arguments (*args)

- Lorsque vous utilisez une fonction, * peut être utilisé pour passer un nombre variable d'arguments positionnels. Cette fonctionnalité est connue sous le nom de déballage d'arguments et est souvent utilisée avec *args :

```
def addition(*args):  
    return sum(args)  
  
result = addition(1, 2, 3, 4) # 10
```

La * dans python

D) Déballage d'arguments nommés (**kwargs)

- L'astérisque double (**) permet de passer un nombre variable d'arguments nommés, généralement sous la forme d'un dictionnaire. Cet usage est appelé **kwargs :

```
def afficher_informations(**kwargs):  
    for key, value in kwargs.items():  
        print(f"{key}: {value}")  
  
afficher_informations(nom="Alice", age=25, ville="Paris")  
# nom: Alice  
# age: 25  
# ville: Paris
```

La * dans python

E) Déballage dans les expression

- * sert à résumer un nombre indéterminé d'arguments

```
first, *middle, last = [1, 2, 3, 4, 5]  
print(middle) # [2, 3, 4]
```

par exemple dans une fonction qui attend 2 arguments, si on dispose des arguments stockés par tuple on peut directement appeler la fonction sans « déballer le tuple » avec deux nouvelles variables

b

```
Debut = edge[0]  
Fin = edge[1]  
print_edge(debut, fin)
```

g

```
print_edge(*edge)
```

...) Les fonctions lambda

- <https://realpython.com/python-lambda/>

Inspiré de la programmation fonctionnelle, une manière d'en faire un peu avec python

Les fonctions lambdas sont comme des mini fonctions qui ne sont pas définies de façon autonome avec `def fonction(x) : ...` comme vous avez l'habitude. Cela sert à aller plus vite pour alléger le code quand ce sont des fonctions « faciles » et de petite taille dont l'usage est unique.

...) Les fonctions lambda

Syntaxe : comme une mini fonction

- | | |
|---------------------------------|----------------|
| Def ajoute_2(x) :
Return x+2 | Lambda x : x+2 |
|---------------------------------|----------------|
-

Une fonction lambda s'exécute ainsi (lambda x: x + 2)(2)

Une expression peut elle même devenir une variable

add_two = lambda x: x + 2

ensuite on peut utiliser la fonction add_two dans le code

...) Les fonctions lambda

Dans la pratique, les fonctions lambdas sont utilisées avec des **méthodes qui prennent une fonction** comme argument

- Map : permet d'appliquer une fonction à tous les éléments d'un itérable
- Filter : applique un filtre sur un itérable et ne conserve que les éléments dont l'application de la fonction donnée en paramètre renvoie True
- Vous pouvez utiliser des fonctions définies avec def comme paramètre de ces fonctions aussi ! Lambda => pour les opérations minimales.

```
>>> list(map(lambda x: x.upper(), ['cat', 'dog', 'cow']))
['CAT', 'DOG', 'COW']
>>> list(filter(lambda x: 'o' in x, ['cat', 'dog', 'cow']))
['dog', 'cow']
```

Et il y en a sans doute encore beaucoup

- Le principal n'est pas de faire énormément de « choses stylées » dans votre code avec des * et des lambdas partout pour frimer, parce que 90 % des gens ne comprendront pas votre code si vous faites cela.
- L'objectif est de comprendre ces animaux quand vous les voyez dans la nature
- Savoir utiliser les plus connus (enumerate, zip, list comprehension) pour écrire un code efficace et clair

<https://www.datacamp.com/tutorial/python-tips-examples>

Autres ressources pour apprendre de nouveaux tricks

- Datacamp
- Medium
- Realpython
- Le github des gens /code source de librairies
- ... votre cours :D



What's the web ?

Historique matériel de l'informatique :

- Années 40 : premiers calculateurs
- Années 50 : premiers ordinateurs commercialisés aux entreprises
- 1977 : premier ordinateur grand public
- 1981 : premier ordinateur « portable »
- 1989 : invention du web

Pourquoi inventer le web ?

Durant la seconde moitié du 20 siècle, les calculateurs puis ordinateurs deviennent de plus en plus disponibles. Ils sont très utiles pour automatiser des tâches mais fonctionnent en silo : chaque machine est isolée et on doit lui fournir les données pour qu'elle puisse réaliser sa tâche. Le développement des réseaux de télécommunications permet matériellement de faire circuler des informations d'une machine à l'autre, mais il faut que la machine dispose des composants pour envoyer et recevoir de l'information.

Internet vs Web

Internet

Réseau mondial d'ordinateurs interconnectés

Né en 1983 (TCP/IP)

Fait circuler **toutes sortes de données** : e-mails, fichiers, messageries, streaming, web...

Fonctionne avec des adresses **IP**

Infrastructure physique + logique (câbles, routeurs, protocoles)

Analogie : **le réseau routier**

Web

Un des services qui fonctionne *sur* Internet

Né en 1989-1991 (Tim Berners-Lee, CERN)

Sert à consulter des pages via des navigateurs

Fonctionne avec des **URL** (http/https://...)

Application de cette infrastructure (sites, hyperliens, HTML)

Analogie : **les voitures qui roulent dessus**

Le Minitel (1982-2012) : un “pré-Web” à la française

- **Origine** : lancé par les PTT (ancêtre d'Orange) en 1982
- **Concept** : un petit terminal offert aux abonnés → connexion via le réseau téléphonique
- **Services** :
 - Annuaire électronique (le 3611)
 - Réservation de billets de train
 - Services bancaires
 - Messageries (ex. 3615 ULLA 🔥)
- **Fonctionnement** : réseau **VidéoTex** → transmission de pages textuelles/graphique basique
- **Succès** : ~9 millions de terminaux en France dans les années 1990
- **Limites** : fermé, centralisé, peu évolutif → remplacé par l'Internet
- **Héritage** : a préparé les usages numériques en France (e-commerce, messageries, info en ligne)



Les adresses IP : la “carte d'identité” des machines sur Internet

- **Définition** : numéro unique attribué à chaque machine connectée à un réseau
- **Rôle** : permet d'acheminer les paquets de données d'un point A à un point B
- **Deux versions principales** :
 - **IPv4** : 32 bits → 4,3 milliards d'adresses (ex. `192.168.0.1`)
 - **IPv6** : 128 bits → quasi illimité (ex. `2001:0db8::1`)
- **Types d'adresses** :
 - Publiques → visibles sur Internet
 - Privées → utilisées en interne (réseaux locaux)
- **Caractéristiques** :
 - Statique → fixe
 - Dynamique → change à chaque connexion
- **Analogie** : comme une adresse postale pour savoir **où envoyer un paquet**

TCP/IP : le langage commun d'Internet

- **Origine** : conçu par **Vint Cerf** et **Robert Kahn** dans les années 1970
- **Mise en service** : 1er janvier 1983 → "flag day" : ARPANET bascule vers TCP/IP
- **Signification** :
 - **TCP** (Transmission Control Protocol) → assure la fiabilité, découpe et recompose les données
 - **IP** (Internet Protocol) → adresse et route les paquets vers leur destination
- **Principe clé** :
 - Communication en **paquets**
 - Chaque machine a une **adresse IP**
 - Le réseau est **décentralisé et robuste**
- **Pourquoi important ?**
 - Base technique d'Internet encore utilisée aujourd'hui
 - Permet l'interopérabilité entre tous les réseaux et machines

La génèse du web :

- Créé par Tim Berners-Lee un scientifique britannique travaillant au CERN en 1989. Cela a commencé comme un problème personnel. Les chercheurs dans les universités utilisaient des calculateurs mais n'avaient pas de moyen de partager leurs données. Le CERN regroupait 17 000 chercheurs dans 100 pays. L'idée de base était de permettre à toutes les machines de fonctionner ensemble, comme cela se fait sur un réseau local, mais délocalisé.

Pour que ce soit possible, les machines doivent suivre un certain nombre de standards pour être capables de communiquer.

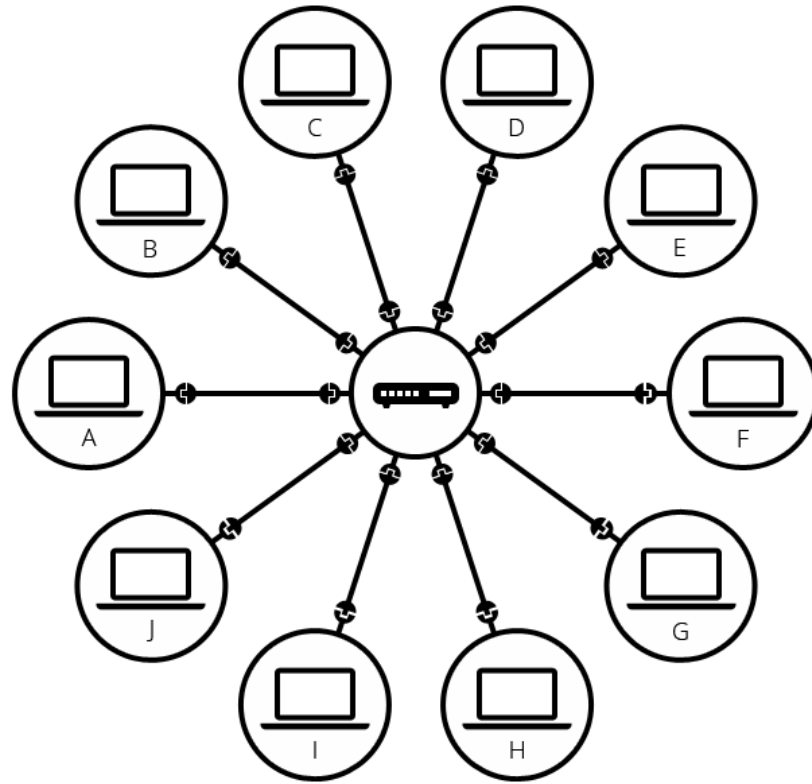
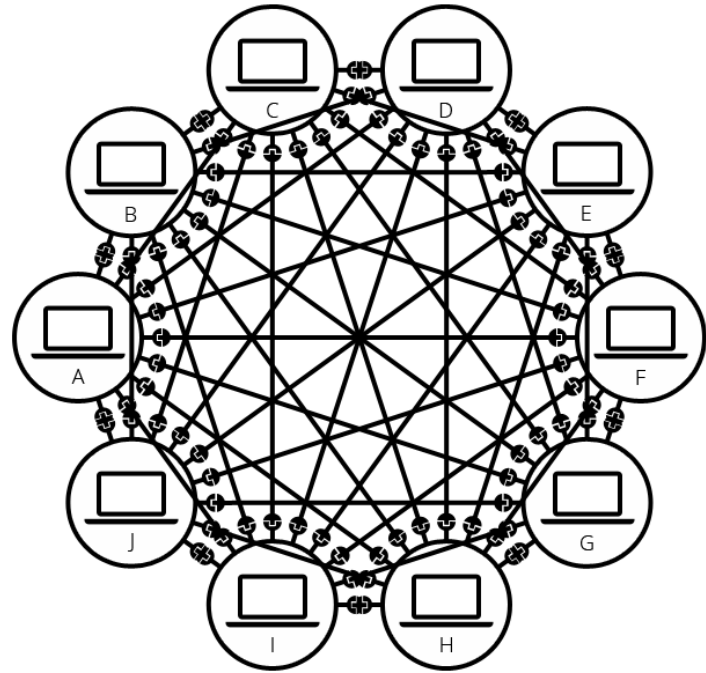
Les premiers pactes (1989/90)

- Avec Robert Cailliau, Tim Berners Lee formalise une proposition pour l'architecture de ce réseau global. Il propose de l'appeler « hypertext project », utilisant la capacité de certains langages de programmation d'augmenter la donnée textuelle par des métadonnées par un travail de balisage.
- L'idée est de créer un réseau global (world wide web) de documents augmentés qui pourraient être consultés à partir d'un navigateur/ explorateur (browser, je traduirais par « parcourer, liseuse »)
- Ces idées de bases décrivent encore la façon dont on utilise internet aujourd'hui, on utilise un programme pour naviguer dans ce réseau de documents.

L'architecture du web

L'architecture du web repose donc sur l'idée qu'avec un programme informatique spécifique (le browser), n'importe quel ordinateur pourrait accéder à des documents.

- Ces documents sont finalement des fichiers.
- Ces fichiers doivent bien être stockés quelque part, et ce n'est donc pas sur la machine qui le consulte.
- Il faut a priori que la machine qui contient / héberge le fichier qu'on veut consulter soit allumée pour pouvoir. Server



- Images de Loïc Grobol

<https://github.com/LoicGrobol/web-interfaces/blob/main/slides/01-internet/internet.py.md>

Liens avec le File system

Chaque page web va être associée à une adresse qui suit la même typographie que le file system (/) et qui permet de trouver le document qu'on veut consulter via le browser.

- L'adresse du fichier (qu'on appelle page web) comporte une partie qui permet de trouver la machine ou le réseau sur lequel se trouve le fichier. La seconde partie de l'adresse décrit sa position précise sur la machine.
- Le premier site au monde était `info.cern.ch`

HTTP : le protocole du Web

- **Signification** : HyperText Transfer Protocol
- **Création** : 1991 par **Tim Berners-Lee** (CERN)
- **Rôle** : protocole de communication entre **navigateur** (client) et **serveur web**
- **Principe** :
 - Le client envoie une **requête HTTP** (ex. `GET /index.html`)
 - Le serveur renvoie une **réponse HTTP** (code + contenu)
- **Exemples de codes de réponse** :
 - `200 OK` → succès
 - `404 Not Found` → page introuvable
 - `500 Internal Server Error` → erreur serveur
- **Évolutions** :
 - HTTP/1.1 (1997) → persistance des connexions
 - HTTP/2 (2015) → plus rapide, multiplexage
 - HTTP/3 (2022) → basé sur QUIC (Google), encore plus performant



Les types de requêtes HTTP

- **GET :**
 - ♦ Récupérer une ressource (page, image, API...)
 - ♦ Lecture seule → ne modifie rien
 - ♦ Exemple : consulter `http://example.com/index.html`
- **POST :**
 - ♦ Envoyer des données au serveur
 - ♦ Utilisé pour **créer** une ressource
 - ♦ Exemple : formulaire de connexion, envoi de commentaire
- **PUT :**
 - ♦ **Remplacer** une ressource existante par une nouvelle
 - ♦ Idempotent (même requête → même résultat)
- **PATCH :**
 - ♦ **Modifier partiellement** une ressource
 - ♦ Plus léger que PUT
- **DELETE :**
 - ♦ Supprimer une ressource

expansion

- Au début, seul le CERN avait un site web et c'était un outil interne, consultable avec un browser maison.
- EN 1991 ils partagent le code du browser
- Stanford a la FOMO et fait pareil, d'autres facs suivent.
- En 1994, le phénomène commence à prendre de l'ampleur.
- Une conférence est formée (International WWW conference Commitee)
- 10 000 serveurs étaient apparus en 4 ans, des fortunes allaient etre faites.